

Model Serving

vLLM · PagedAttention · Continuous Batching · TGI

Prefix Caching · Speculative Decoding · Multi-GPU · TTFT vs Throughput

WEEK 2 · DAY 11

Thursday, July 16 2026

Day 11 of 84

#	Topic	What You Learn
01	The Serving Problem	Why naive inference is slow; memory bandwidth bottleneck
02	Continuous Batching	Iteration-level scheduling; 5-10x throughput vs static batch
03	PagedAttention	Virtual memory for KV cache; 97% GPU utilization
04	vLLM Setup	Installation, config params, AWQ quantized serving
05	TGI — HuggingFace	Docker deployment; vLLM vs TGI decision guide
06	Throughput vs Latency	TTFT, TPOT, prefill vs decode phase
07	Prefix Caching	Eliminate redundant system prompt computation
08	Speculative Decoding	Draft model + verifier; 1.5-3x speedup on SQL/code
09	Multi-GPU Strategies	Tensor, pipeline, data parallelism for serving
10	FAANG in Production	Meta, Google, Amazon, Netflix, Microsoft serving stacks
11	Text-to-SQL Thread	Serving the fine-tuned SQL model end-to-end
12	Architect's Lens	Decision tree: vLLM vs TGI, TP vs DP, production targets

TEXT-TO-SQL CAPSTONE THREAD — Day 11 closes the inference loop: your fine-tuned SQL model (Day 10) is quantized (Day 8), served via vLLM with prefix caching on the schema system prompt, and exposed as an OpenAI-compatible API ready for Week 3 RAG integration.

Section 1 — Why Naive Inference Is Slow

You have a fine-tuned model. A naive serving loop calls `model.generate()` one request at a time — your GPU sits idle 90% of the time. Understanding why requires understanding the autoregressive bottleneck.

The Autoregressive Generation Problem

LLMs generate one token at a time. Each token requires a full forward pass through all N layers. For a 1,000-token response: 1,000 sequential forward passes.

GPU utilization during single-request generation:

- Attention computation: ~15% of time
- FFN computation: ~25% of time
- Memory transfers (HBM): ~60% of time <- the real bottleneck

Why memory-bound?

- LLaMA 3 8B in BF16 = 16 GB of weights
- Every generated token reads the FULL 16 GB from HBM (High Bandwidth Memory)
- A100 HBM bandwidth: 2 TB/s
- Time to read 16 GB: $16 / 2000 = 8\text{ms}$ minimum per token
- > Max ~125 tokens/second per request, regardless of compute speed

Static Batching — The Naive Fix

Batch requests together: one forward pass serves multiple requests. Problem: the batch must wait for the LONGEST sequence to finish before any completed requests can be replaced.

Sequence 1:  (done at token 80, waiting idle until 200)
 Sequence 2:  (longest — determines batch end at token 200)
 Sequence 3:  (done at 120, waiting)
 Sequence 4:  (done at 40, longest wait)
 <- grey boxes = wasted GPU capacity ->

GPU utilization with static batching: 35-55% on variable-length workloads

Section 2 — Continuous Batching (Iteration-Level Scheduling)

Continuous batching (Orca, 2022) changes scheduling granularity from request-level to iteration-level (one step = one token). A completed sequence is immediately replaced with a new request — the GPU never waits. The batch is always full.

Iteration (token step) timeline:

Step 1: [Req A tok 1] [Req B tok 1] [Req C tok 1] [Req D tok 1]
 Step 2: [Req A tok 2] [Req B tok 2] [Req C tok 2] [Req D tok 2]

```
Step 3: [Req A tok 3] [Req B tok 3] [Req C tok 3] [Req E tok 1] <- D done; E joins
Step 4: [Req A tok 4] [Req B tok 4] [Req C tok 4] [Req E tok 2]
Step 5: [Req A tok 5] [Req F tok 1] [Req C tok 5] [Req E tok 3] <- B done; F joins
```

The batch is always full. GPU utilization rises to 75-90%.
Result: 5-10x throughput improvement over static batching.

Section 3 — PagedAttention: vLLM's Core Innovation

Continuous batching solved scheduling. But it created a new problem: KV cache memory fragmentation. Each sequence in the batch must store key-value vectors for all tokens it has generated so far.

The KV Cache Memory Problem

KV cache size per sequence (LLaMA 3 8B, BF16, max 4096 tokens):

```
= 2 (K and V)
* 32 layers
* 32 heads
* 128 head_dim
* 4096 tokens
* 2 bytes (BF16)
= 2,147,483,648 bytes = 2.15 GB per sequence
```

For 8 concurrent sequences (naive contiguous allocation):

```
Model weights:    16 GB
8x KV caches: +  17.2 GB
Total:            33.2 GB <- fits on A100 80GB barely, but with fragmentation
                        most of the 17.2 GB is UNUSED reserved space
                        (sequences shorter than 4096 tokens)
```

Memory fragmentation with contiguous allocation: 40-60% wasted KV memory

PagedAttention — Virtual Memory for KV Cache

PagedAttention (Kwon et al., 2023) divides KV cache into fixed-size blocks (pages). Each sequence's KV cache is stored in non-contiguous blocks, linked by a block table — identical to OS virtual memory paging.

Physical GPU memory (with PagedAttention, block size = 16 tokens):

```
Block 0: [Seq A tok 1-16 K/V]    Block 4: [Seq B tok 17-32 K/V]
Block 1: [Seq B tok 1-16 K/V]    Block 5: [FREE]
Block 2: [Seq A tok 17-32 K/V]   Block 6: [Seq A tok 33-48 K/V]
Block 3: [Seq C tok 1-16 K/V]    ...
```

Block table (virtual -> physical mapping):

```
Seq A: [0, 2, 6, ...] <- non-contiguous; allocated on demand as tokens generated
Seq B: [1, 4, ...]
Seq C: [3, ...]
```

Result: Near-zero internal fragmentation.

Only USED tokens consume KV cache memory.

vLLM achieves 97-99% GPU memory utilization for KV cache.

Throughput Comparison

Serving Method	Concurrency	Tokens/sec (8B, A100)
Naive single-request	1	~150
Static batching (batch=8)	8	~450
Continuous batching (vLLM, no PA)	32	~2,100
Continuous batching + PagedAttn	128+	~4,200

Section 4 — vLLM: The Production Standard

vLLM (UC Berkeley, 2023) implements continuous batching + PagedAttention + Flash Attention 2 + tensor parallelism. It serves an OpenAI-compatible API — any code using the OpenAI SDK works with vLLM with one URL change.

Installation and Basic Serving

```
pip install vllm # includes torch, flash-attn dependencies

# Serve via OpenAI-compatible API:
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3.1-8B-Instruct \
    --dtype bfloat16 \
    --tensor-parallel-size 1 \           # GPUs to use
    --gpu-memory-utilization 0.90 \    # 90% of GPU VRAM for KV cache
    --max-model-len 4096 \
    --port 8000

# Client (identical to OpenAI SDK usage):
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="unused")
resp = client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=[{"role": "user", "content": "Write SQL to count users by country"}],
    max_tokens=512, temperature=0.1,
)
print(resp.choices[0].message.content)
```

Key Configuration Parameters

```
from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    dtype="bfloat16",
    tensor_parallel_size=1,          # 1 for 8B; 4 for 70B full BF16
    gpu_memory_utilization=0.90,    # fraction for KV cache; raise for more concurrency
    max_model_len=4096,             # max context length
    quantization="awq",              # serve AWQ model (optional, Day 8)
    swap_space=4,                   # CPU RAM fallback for KV cache overflow (GB)
    max_num_seqs=256,               # max concurrent sequences in flight
    enable_prefix_caching=True,     # cache KV for repeated system prompts
)

sampling = SamplingParams(
    temperature=0.1,
    top_p=0.9,
    max_tokens=512,
    stop=["``", "\n\n"],
)
```

```
outputs = llm.generate(["Write SQL to count users by country"], sampling)
print(outputs[0].outputs[0].text)
```

Serving a Quantized Model (AWQ)

```
# LLaMA 3 70B memory requirements:
# Full BF16:      140 GB  -> 2x A100 80GB minimum (TP=2)
# AWQ INT4:       35 GB  -> 1x A100 80GB (model + KV cache fits)
# GPTQ INT4:      35 GB  -> 1x A100 80GB

# Serve AWQ LLaMA 3 70B on 1x A100 80GB:
python -m vllm.entrypoints.openai.api_server \
  --model TheBloke/Llama-3-70B-Instruct-AWQ \
  --quantization awq \
  --dtype float16 \           # AWQ requires float16 (not bfloat16)
  --tensor-parallel-size 1 \  # fits on 1 GPU after quantization
  --gpu-memory-utilization 0.85 \
  --max-model-len 8192 \
  --port 8000
```

Section 5 — TGI: HuggingFace Text Generation Inference

TGI is HuggingFace's production serving engine — the same core algorithms (continuous batching + PagedAttention) as vLLM, with built-in Prometheus metrics and native HuggingFace Inference Endpoints support.

```
# Docker deployment (production standard):
docker run --gpus all \
  -p 8080:80 \
  -v ~/.cache/huggingface:/data \
  ghcr.io/huggingface/text-generation-inference:latest \
  --model-id meta-llama/Llama-3.1-8B-Instruct \
  --dtype bfloat16 \
  --max-input-length 4096 \
  --max-total-tokens 8192 \
  --max-batch-prefill-tokens 16384

# Client:
from huggingface_hub import InferenceClient
client = InferenceClient(base_url="http://localhost:8080")
resp = client.text_generation(
    "Write SQL to count users by country",
    max_new_tokens=512, temperature=0.1, stop_sequences=["``"],
)
print(resp)
```

vLLM vs TGI — Decision Guide

Dimension	vLLM	TGI
-----------	------	-----

PagedAttention	Yes (original implementation)	Yes (adopted 2024)
Continuous batching	Yes	Yes
OpenAI API compat	Native (/v1/chat/completions)	Via adapter
Model support	Broader (more architectures)	HuggingFace ecosystem
Prometheus metrics	Basic	Built-in, production-ready
Quantization	AWQ, GPTQ, INT8, FP8	AWQ, GPTQ, INT8
HF Inference Endpoints	Not supported	Native (managed)
Best for	Custom deploy, research, speed	HF Endpoints, enterprise

RULE: Start with vLLM for fastest iteration and maximum control. Switch to TGI when deploying on HuggingFace Inference Endpoints, when you need Prometheus metrics out of the box, or when your team is already embedded in the HuggingFace ecosystem.

Section 6 — Throughput vs Latency: The Fundamental Trade-off

THROUGHPUT: tokens generated per second (system-level metric)

LATENCY: time to first token + time per output token (request-level metric)

The trade-off:

Large batch size -> high throughput, high latency (each request waits longer)

Small batch size -> low throughput, low latency (GPU less utilized)

Benchmark (LLaMA 3 8B, A100 80GB, 512-token prompt, 200-token output):

batch=1: TTFT=85ms, throughput=180 tok/sec

batch=8: TTFT=120ms, throughput=720 tok/sec

batch=32: TTFT=280ms, throughput=2,100 tok/sec

batch=128: TTFT=890ms, throughput=4,800 tok/sec

TTFT and TPOT

TTFT – Time To First Token:

How long until the user sees the first word.

= prefill time (process entire prompt in one parallel pass)

Target: < 200ms P50 for interactive use, < 500ms P95

TPOT – Time Per Output Token:

Time between each subsequent token.

= decode time per step (memory-bandwidth-bound)

Target: < 50ms P50, < 100ms P95

At 50ms/token: 20 tok/sec = ~3 words/sec (human reading pace)

Total latency for a 200-token response (streaming):

= TTFT + (200 tokens * TPOT)

= 120ms + (200 * 35ms) = 7.1 seconds total

Without streaming: user waits 7.1 seconds

With streaming: user sees first word at 120ms; tokens appear progressively

Prefill vs Decode Phase

Phase	What Happens	Bound By	How to Optimize
Prefill	All prompt tokens processed in ONE parallel pass	Compute (arithmetic)	Flash Attention 2, longer context = more cost
Decode	ONE token generated per forward pass	Memory bandwidth (HBM read)	Larger batches, speculative decoding, quantization

For SQL generation with 2K-token schema context (long prompt): prefill dominates TTFT — invest in Flash Attention 2 and prefix caching. For chat with 50-token prompts: decode dominates — invest in larger batch sizes and speculative decoding.

Section 7 — Prefix Caching: Eliminating Redundant Prefill

Most production systems send the same system prompt with every request. Without caching, every request re-computes the KV cache for the system prompt. Prefix caching detects shared prefixes and reuses their KV cache blocks.

```
# Enable in vLLM:
llm = LLM(model="...", enable_prefix_caching=True)

# System prompt (identical for all SQL generation requests):
SCHEMA_SYSTEM = """You are a SQL expert. Given a schema and question,
generate correct SQL. Use aliases. No SELECT *. SQL only.""" # ~50 tokens

# Request 1: SCHEMA_SYSTEM KV cache computed and stored in blocks
# Request 2: SCHEMA_SYSTEM KV cache REUSED -> zero prefill cost for those tokens
# Request 3: REUSED again

# With 256-token system prompt at 1M requests/day:
# Without prefix cache: 256M tokens prefilled per day
# With prefix cache:    256 tokens prefilled ONCE (amortized to ~0)
# TTFT improvement: 280ms -> 85ms (system prompt no longer re-processed)

# Netflix scale (100M requests/day, 512-token system prompt):
# Prefill savings: 100M * 512 * 0.008ms/token = ~40M GPU-ms saved per day
# = ~$15,000/day in A100 compute costs
```

TEXT-TO-SQL: The schema system prompt (table names, column types, SQL rules) is typically 200-500 tokens — constant across all user queries. Prefix caching makes this essentially free after the first request. At 10,000 SQL queries/day, prefix caching saves 2-5 A100-hours per day.

Section 8 — Speculative Decoding

Decode phase is sequential and memory-bound — one token per step, each reading the full model from HBM. Speculative decoding uses a small draft model to propose several tokens at once, then verifies all of them in one pass with the large model.

Standard decoding (LLaMA 3 70B, 1 token/step):

```
Step 1: [70B forward pass] -> token 1    (8ms)
Step 2: [70B forward pass] -> token 2    (8ms)
...
10 tokens: 10 x 8ms = 80ms
```

Speculative decoding (1B draft + 70B verifier):

```
Draft: [1B forward pass x5] -> propose tokens 1,2,3,4,5 (~1ms total)
Verify: [70B forward pass x1, all 5 tokens parallel]      (8ms)
If all 5 accepted: 5 tokens in 9ms instead of 40ms -> 4.4x speedup
If 3 accepted:      3 tokens in 9ms; resample token 4 from 70B
```

Key property: output distribution is MATHEMATICALLY IDENTICAL to 70B alone (rejection sampling guarantee - speculative decoding never changes quality)

Expected acceptance rate by task:

```
SQL / code generation:      80-90% (highly predictable, low entropy)
Structured JSON output:     85-95% (constrained, very predictable)
Creative writing:            50-65% (high entropy, draft often wrong)
General chat:               65-75% (moderate entropy)
```

Enable in vLLM:

```
llm = LLM(
    model="meta-llama/Llama-3.1-70B-Instruct",
    speculative_model="meta-llama/Llama-3.2-1B-Instruct", # draft model
    num_speculative_tokens=5, # draft this many tokens per step
    tensor_parallel_size=4,
)
# All other code unchanged - same SamplingParams, same outputs format
# Typical speedup on SQL tasks: 1.5-2.5x
# Memory overhead: +2GB for the 1B draft model
```

Section 9 — Multi-GPU Serving Strategies

Tensor Parallelism (TP) — Split Each Layer Across GPUs

LLaMA 3 70B across 4x A100 80GB:

```
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Llama-3.1-70B-Instruct \
    --tensor-parallel-size 4 \ # split each attention/FFN matrix across 4 GPUs
    --dtype bfloat16 \
    --gpu-memory-utilization 0.85
```

Memory per GPU: 140 GB / 4 = 35 GB weights + ~30 GB KV cache = 65 GB (fits)

Requires NVLink for low-latency GPU-GPU communication
Best for: minimizing LATENCY (all 4 GPUs collaborate on every request)

Data Parallelism (DP) — Replicate for Throughput

Run N independent model replicas; load balancer distributes requests:
GPU 0-1: replica 1 (each handles different requests)
GPU 2-3: replica 2
Load balancer: round-robin or latency-aware routing

Best for: model fits on 1-2 GPUs; want to maximize total throughput
Latency stays low (each replica handles a smaller sub-batch)

Model Size vs GPU Configuration

Model	Full BF16 Memory	Recommended Config	Fits On
LLaMA 3 8B	16 GB	TP=1, BF16	1x A100 40GB
LLaMA 3 13B	26 GB	TP=1, BF16 or AWQ	1x A100 40GB (AWQ)
LLaMA 3 70B	140 GB	TP=4, BF16 OR TP=1, AWQ	4x A100 80GB or 1x A100 80GB (AWQ)
LLaMA 3 405B	810 GB	TP=8, AWQ	8x A100 80GB with AWQ

ARCHITECT RULE: Tensor parallelism minimizes latency (all GPUs work on every request simultaneously). Data parallelism maximizes throughput (each GPU serves different requests independently). For real-time interactive serving: TP. For batch / async high-volume serving: DP. Production deployments use both.

Section 10 — FAANG in Production

Meta — LLaMA Serving at Scale

Meta serves the public Meta AI assistant (WhatsApp, Instagram, Messenger) on LLaMA 3 models. Key architectural choices: Tensor parallel across 8 H100s per replica for low latency real-time chat; prefix caching for the large Meta AI personality system prompt shared across billions of requests; mixed model routing — LLaMA 3 8B for simple replies, 70B for complex queries, router selects based on query complexity score. Target: 50,000 tokens/second sustained throughput across the fleet.

Google — Cascade Serving + TPU Inference

Google's Gemini serving runs on TPUs via JAX with their internal Pathways serving system. Innovation: Cascade serving — requests that can be answered quickly stay with a small model; genuinely difficult queries escalate to large model. This is speculative decoding applied at the model level rather than token level. For external customers on Vertex AI: NVIDIA Triton Inference Server. TPU advantage: 3x cheaper inference per token vs A100 for Google's own workloads.

Amazon — SageMaker TGI + Bedrock

Amazon Bedrock serves Claude and other FMs via SageMaker endpoints. For LLaMA 3, AWS provides pre-built TGI containers via HuggingFaceModel class. Standard instance: ml.g5.12xlarge (4x A10G GPUs, 96 GB VRAM). SageMaker handles auto-scaling, model caching between invocations, and multi-model endpoints (multiple models sharing one GPU fleet). Bedrock adds: content filtering as serving-layer middleware, per-customer rate limiting, CloudWatch metrics for latency and throughput.

Netflix — Sub-100ms Serving for Recommendations

Netflix's real-time paths require P95 TTFT < 100ms — recommendation results must appear before the user finishes scrolling. Achieves this with: Haiku-class models only (LLaMA 3 8B or Gemma 2B) for real-time paths; aggressive prefix caching — catalog metadata system prompt is 8K tokens, cached permanently; streaming response for all user-facing text generation. Async batch inference (TGI on SageMaker) for content tagging and A/B analysis runs at night when GPU fleet is underutilized.

Microsoft — Azure OpenAI Service Architecture

Azure OpenAI Service (ChatGPT Enterprise backend) uses a modified vLLM with Azure additions: per-customer rate limiting enforced at the serving layer, Azure Content Safety as middleware, multi-region routing for latency and availability. Architecture: TP=8 across A100s per model instance, data-parallel across Azure regions. Microsoft's Phi-3 models are served via the same stack — the small model size (3.8B) means TP=1 on a single A10G, making it very cost-efficient for Azure customers.

Section 11 — Text-to-SQL Thread: Serving the Fine-Tuned Model

Day 10: full fine-tuned + merged LLaMA 3 8B SQL model. Here is the complete Day 11 serving pipeline closing the inference loop:

```
# Step 1: Model from Day 10 lives at ~/models/sql-llama-8b-merged/
# Step 2: Quantize to AWQ for 2x memory savings (Day 8 technique):
python -c "
from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer
model = AutoAWQForCausalLM.from_pretrained('~/models/sql-llama-8b-merged')
tokenizer = AutoTokenizer.from_pretrained('~/models/sql-llama-8b-merged')
model.quantize(tokenizer, quant_config={
    'zero_point': True, 'q_group_size': 128, 'w_bit': 4, 'version': 'GEMM'
})
model.save_quantized('~/models/sql-llama-8b-awq')
"

# Memory: 16 GB (BF16) -> 4.5 GB (AWQ INT4) + overhead = ~6 GB total
# Fits comfortably on A100 80GB; even fits on RTX 3090 (24GB)

# Step 3: Serve with vLLM + prefix caching:
python -m vllm.entrypoints.openai.api_server \
    --model ~/models/sql-llama-8b-awq \
    --quantization awq \
    --dtype float16 \
    --gpu-memory-utilization 0.85 \
    --max-model-len 8192 \
    --enable-prefix-caching \ # cache the schema system prompt
    --port 8000

# Step 4: Production client with prefix caching:
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="unused")

SCHEMA_SYSTEM = """You are a SQL expert for a retail analytics database.
Schema:
    orders(order_id, user_id, amount, created_at, status)
    users(user_id, email, country, created_at)
    products(product_id, name, category, price)
    order_items(order_id, product_id, quantity, unit_price)
Rules: Use table aliases. Never SELECT *. Output only valid SQL."""

def text_to_sql(question: str) -> str:
    resp = client.chat.completions.create(
        model="sql-llama-8b-awq",
        messages=[
            {"role": "system", "content": SCHEMA_SYSTEM}, # cached after first call!
            {"role": "user", "content": question},
        ],
        max_tokens=512,
```

```

        temperature=0.05,    # near-deterministic for SQL
        stop=[";", "+", "\n\n", "```"],
    )
    return resp.choices[0].message.content

```

```

# Performance with prefix caching enabled:
# Request 1 (cold): TTFT ~120ms (schema computed and cached)
# Request 2+ (warm): TTFT ~35ms (schema KV reused)
# Throughput: ~3,500 SQL tokens/second on 1x A100 with AWQ

```

Section 12 — Architect's Lens

SERVING STACK DECISION TREE

Q1: Need OpenAI API compatibility?

```

YES -> vLLM (native) or TGI (via adapter)
NO  -> either, or Triton for NVIDIA ecosystem

```

Q2: Primary deployment target?

```

HuggingFace Inference Endpoints -> TGI (native support)
AWS SageMaker                    -> TGI (pre-built containers)
Custom Kubernetes / bare metal  -> vLLM (more control)
NVIDIA Triton ecosystem         -> Triton Inference Server

```

Q3: What is your primary constraint?

THROUGHPUT (batch, async):

```

-> max_num_seqs=256, gpu_memory_utilization=0.90
-> Enable speculative decoding (1.5-3x for SQL/code)
-> Data-parallel replicas for horizontal scale

```

LATENCY (real-time, streaming):

```

-> Streaming API, small batches
-> Enable prefix caching for system prompts
-> Tensor parallel if model needs multiple GPUs
-> Target: TTFT < 200ms P50, TPOT < 50ms P50

```

Q4: Model size vs available GPUs?

```

8B  on 1x A100 80GB: TP=1, BF16 or AWQ, ~4,000 tok/sec
70B on 1x A100 80GB: TP=1, AWQ INT4 required
70B on 4x A100 80GB: TP=4, BF16 (preferred for quality)
405B on 8x A100 80GB: TP=8, AWQ required

```

PRODUCTION TARGETS (what good looks like):

```

TTFT P50:          < 200ms (interactive)
TTFT P95:          < 500ms (interactive)
TPOT P50:          < 50ms  (streaming visible at human reading pace)
GPU utilization:    > 75%   (sustained)
Throughput (8B, 1xA100): 2,000-5,000 tokens/second

```


Section 13 — Hands-On Exercises

Exercise 1: Async Throughput Benchmark

```
import time, asyncio
from openai import AsyncOpenAI

client = AsyncOpenAI(base_url="http://localhost:8000/v1", api_key="unused")

async def single_request(prompt):
    start = time.perf_counter()
    resp = await client.chat.completions.create(
        model="meta-llama/Llama-3.1-8B-Instruct",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=200,
    )
    return time.perf_counter() - start, resp.usage.completion_tokens

async def benchmark(n_concurrent=10, n_total=50):
    prompts = [f"Write SQL for query #{i}: count orders by status" for i in range(n_total)]
    sem = asyncio.Semaphore(n_concurrent)

    async def bounded(p):
        async with sem:
            return await single_request(p)

    start = time.perf_counter()
    results = await asyncio.gather(*[bounded(p) for p in prompts])
    total_time = time.perf_counter() - start
    total_toks = sum(r[1] for r in results)
    lats = sorted(r[0]*1000 for r in results)

    print(f"Concurrency={n_concurrent} | {total_toks/total_time:.0f} tok/s "
          f"| P50={lats[n_total//2]:.0f}ms | P95={lats[int(n_total*.95)]:.0f}ms")

    asyncio.run(benchmark(n_concurrent=1))    # baseline
    asyncio.run(benchmark(n_concurrent=16))  # throughput mode
    asyncio.run(benchmark(n_concurrent=64))  # max throughput
```

Exercise 2: Measure Prefix Cache Speedup

```
import time
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="unused")

LONG_SYSTEM = "You are a SQL expert for a complex retail schema. " * 50 # ~500 tokens

def measure_ttft(with_system: bool) -> float:
    msgs = []
    if with_system:
        msgs.append({"role": "system", "content": LONG_SYSTEM})
    msgs.append({"role": "user", "content": "Write SQL to count users"})
```

```

start = time.perf_counter()
with client.chat.completions.create(
    model="meta-llama/Llama-3.1-8B-Instruct",
    messages=msgs, max_tokens=80, stream=True,
) as stream:
    for chunk in stream:
        if chunk.choices[0].delta.content:
            return (time.perf_counter() - start) * 1000

# Warm up cache with first request, measure second:
print(f"With system (cold): {measure_ttft(True):.0f}ms")
print(f"With system (warm): {measure_ttft(True):.0f}ms") # should be faster
print(f"Without system: {measure_ttft(False):.0f}ms")

```

Exercise 3: Choose the Right Serving Config

Scenario	Best Config
70B model, minimize cost, 1s latency OK	AWQ INT4 + 1x A100 80GB + large batches (TP=1)
8B model, real-time chat, P95 < 300ms	TP=1, prefix caching, streaming, batch size small
8B model, 100K req/day SQL, async	TP=1 BF16 + speculative decoding + enable_prefix_caching
Air-gapped deployment, no GPU	GGUF Q4_K_M + llama.cpp on CPU (not vLLM)
HuggingFace Inference Endpoint	TGI (native, managed infrastructure)

Summary — Day 11 Key Concepts

Concept	One-Line Definition
Static batching	Wait for all sequences to finish before replacing any
Continuous batching	Replace finished sequences immediately at each token step
PagedAttention	KV cache in non-contiguous pages; ~0% memory fragmentation
TTFT	Time To First Token — prefill phase latency
TPOT	Time Per Output Token — decode phase bandwidth limit
Prefill phase	All prompt tokens in ONE parallel forward pass (compute-bound)
Decode phase	One token per forward pass (memory-bandwidth-bound)
Prefix caching	Reuse KV blocks for shared system prompts across requests
Speculative decoding	Draft model proposes; large model verifies; identical output
Tensor parallelism	Split weight matrices across GPUs; minimizes latency
Data parallelism	Replicate full model; maximizes throughput
vLLM	UC Berkeley serving engine; PagedAttn + continuous batch + OpenAI API
TGI	HuggingFace serving engine; Prometheus metrics; HF Endpoints native

TOMORROW — Day 12: Evaluation

Your model is trained (Days 8-10) and served (Day 11). Day 12 answers: is it actually good? MMLU, HumanEval, MT-Bench, LLM-as-judge, execution accuracy for SQL. How to build a rigorous evaluation harness that catches regressions before they reach production.

Eagle Eye Addendum — Day 11: FastAPI Serving Layer

FastAPI is the standard Python web framework for building the HTTP serving layer on top of an LLM backend (vLLM, Triton, etc.). It provides OpenAPI docs, Pydantic validation, and async support.

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import httpx

app = FastAPI(title="LLM Serving API")

class GenerateRequest(BaseModel):
    prompt: str
    max_tokens: int = 256
    temperature: float = 0.7
    tenant_id: str = "default"

class GenerateResponse(BaseModel):
    text: str
    tokens_used: int
    latency_ms: float

VLLM_URL = "http://localhost:8000/v1/completions"

@app.post("/generate", response_model=GenerateResponse)
async def generate(req: GenerateRequest):
    import time; t0 = time.perf_counter()
    async with httpx.AsyncClient() as client:
        r = await client.post(VLLM_URL, json={
            "model": "meta-llama/Meta-Llama-3-8B-Instruct",
            "prompt": req.prompt,
            "max_tokens": req.max_tokens,
            "temperature": req.temperature,
        })
    if r.status_code != 200:
        raise HTTPException(status_code=502, detail="vLLM backend error")
    data = r.json()
    return GenerateResponse(
        text = data["choices"][0]["text"],
        tokens_used= data["usage"]["total_tokens"],
        latency_ms = (time.perf_counter() - t0) * 1000)

# Run: uvicorn serve:app --host 0.0.0.0 --port 9000 --workers 4
# FastAPI wraps vLLM and adds: auth, rate limiting, logging, Prometheus metrics.
```